Lecture 01: Introduction to Backend and Creating a Server

First-Class Citizens in JavaScript

Objects are the first-class citizens in JavaScript because almost everything under the hood is fundamentally an object (including arrays, functions, and DOM elements).

Understanding the Need of Backend

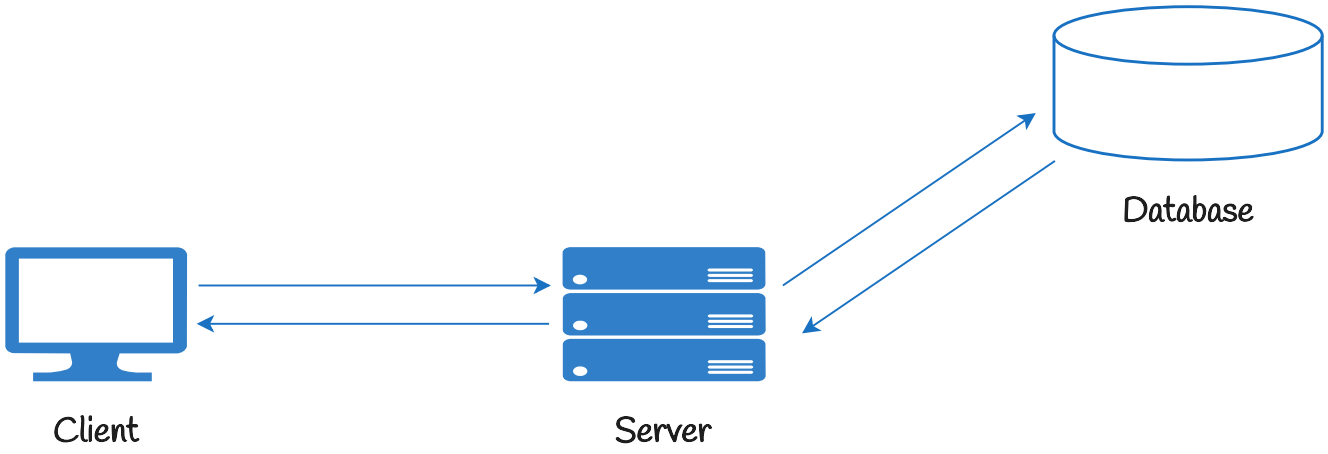
We can best understand the need for a backend by looking at how we store data across different stages of application development:

- **Project 1 (Simple Frontend):** Data is stored in the browser's RAM. If the page is refreshed, the RAM is cleared, and all data is permanently lost.
- **Project 2 (Secondary Storage):** Data is saved to secondary storage, making it permanent. However, this data is locked to one specific device. A profile logged into a different device will not see this data. Additionally, strict security rules prevent one application from reading another application's local storage.
- **Project 3 (Shared Cloud Database):** Data is stored centrally in a database.

Real-World Analogy: Just like uploading a file to Google Drive makes it accessible 24/7 from anywhere, a backend database ensures data can be fetched continuously from any device or profile.

Security and the Credential Exposure Problem

- Frontend code is entirely visible through browser developer tools. Hardcoding database URLs and access keys directly into the frontend exposes admin credentials, allowing anyone to modify or delete data.
- **The Solution:** The backend acts as a secure intermediary.
- The frontend communicates only with the backend, and only the backend holds the database access credentials.



How the Internet Works (Behind the Scenes)

To communicate with another server, a device must know that server's IP address.

- **DNS (Domain Name System):** When a URL is requested, a DNS server translates that URL into the exact IP address of the target server.
- **ISP (Internet Service Provider):** The ISP provides both the local IP address and the default DNS address. If the internet disconnects, the assigned IP address may change upon reconnection.
- **Custom DNS:** The default DNS can be changed manually to providers like Google (8.8.8.8) or Cloudflare (1.1.1.1).

Port Numbers and Data Packets

Data is transferred across the internet in binary form. When data reaches the target device, the Operating System must determine exactly which application should receive it. Port Numbers are used for this exact identification.

- **Packet Structure:**

Source IP	Source Port	Data	Destination Port	Destination IP

- Port numbers are 16-bit (2 Bytes) and range from 0 to 65,535.
- **Default Ports:** Standard protocols have fixed ports: HTTP uses 80, and HTTPS uses 443.
- **HTTP vs. HTTPS:** HTTPS is encrypted, meaning only the receiving server has the ability to decrypt and read the data.

Building the First Server (Node.js)

- A server is simply code that continuously listens for incoming requests on a specific port and returns responses.
- Node.js uses the built-in http module (a wrapper over C++ code) to create servers.

```
const http = require('http');

const server = http.createServer((request, response) => {
  response.end("Hello Coder Army");
});

server.listen(3000,() => {
  console.log("Server is listening at port 3000");
});
```

Local Servers, Routing, and JSON Data

- **Localhost (http://localhost:3000):** During local development, the frontend, backend, and database all run on a single machine. Using localhost:3000 skips DNS resolution entirely and connects directly to port 3000 on that machine.
- **The Request Object:** The incoming request is structured as an object. The request.url property holds the exact path requested by the client.
- **Parsing the URL String:** The number can be extracted by looping through the request.url string and converting it into a Number.
- **Data Format:** Backends cannot return raw JavaScript arrays or objects directly to the client. The data must be converted into a string format using JSON.stringify().

```
const http = require('http');

const github = [
];

const server = http.createServer((request, response) => {
   let str = request.url;
   let numStr = "";
   for (let i = 1; i < str.length; i++) {
      numStr += str[i];
   }

   let number = Number(numStr);

   const arr = [];

   for (let i = 0; i < number; i++) {
      arr.push(github[i]);
   }

   response.end(JSON.stringify(arr));
});

server.listen(3000, () => {
   console.log("Server is listening at port 3000");
});
```

Note: Whenever backend code is modified, the server must be manually stopped and restarted to execute the fresh code.